

SIMATS ENGINEERING ASSESSMENT TOOL

COURSE NAME: Theory of Computation

COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ϵ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 10%

QUESTIONS: 5 Total Marks:25 Duration : 1 Hour

Application Based Question

Code of Conduct:

I J.Goutham Kumar Reddy / 192311140 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING ASSESSMENT TOOL

1. In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable. Design a **Deterministic Finite Automaton (DFA)** over the alphabet $\Sigma = \{0,1\}$ to validate passwords that **end with the pattern "01"**. Further, **minimize the DFA** and analyze how the optimized design improves system efficiency in real-world hardware implementations

ANS)

Alphabet

$$\Sigma = \{0,1\} \setminus \Sigma = \{0,1\} \setminus \Sigma = \{0,1\}$$

The DFA accepts all binary passwords that end with the pattern "01".

States

- q_0 – Start state
- q_1 – Last input symbol is 0
- q_2 – Last two symbols are 01 (Accept state)

Transition Table

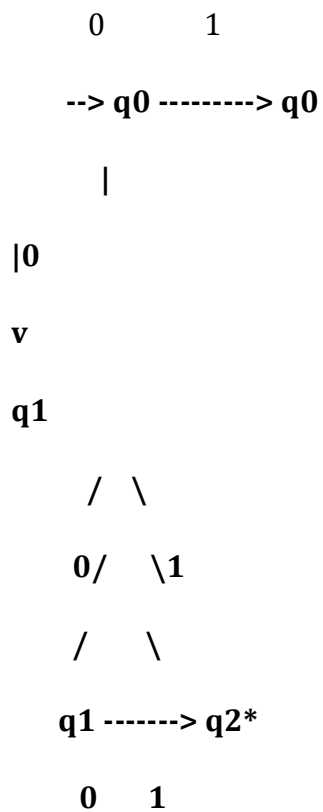
Present State	Input 0	Input 1
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_1	q_0

q_0 • Start

State: q_0

- Final State: q_2

State Diagram



From q2:

$0 \rightarrow q1$

$1 \rightarrow q0$

DFA Minimization

The DFA has 3 states. After applying DFA minimization, no two states are equivalent, so no states can be merged.

Minimized DFA = 3 states

.

2. In web applications, validating user input is essential to ensure correctness, security, and performance. Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet $\Sigma = \{a, b\}$. Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications.

ANS)

Alphabet

$\Sigma = \{a, b\}$ \Sigma = \{a, b\}

The required input pattern is "aaaabb", which contains exactly four a's followed by two b's.

Regular Expression

$^{\wedge}$ aaaabb\$ or

simply

aaaabb

- $^{\wedge}$ indicates the beginning of the string.
- **aaaabb** matches exactly four a's followed by two b's.
- \$ indicates the end of the string.

This regular expression accepts only the string "aaaabb".

Examples

Input	Valid/Invalid
-------	---------------

aaaab b	Valid
------------	-------

aaabb	Invalid
-------	---------

aaaab	Invalid
-------	---------

aaaab bb	Invalid
-------------	---------

baaab b	Invalid
------------	---------

Use of Regular Languages and Regular Expressions in Web Applications

Regular languages and regular expressions are widely used in web applications for input validation because they are simple, fast, and efficient.

Some common applications include:

- Email validation (e.g., [user@example.com](#))
- Phone number validation
- Password format checking
- PIN or OTP verification
- Username validation (allowing only specific characters)

Regular expressions help ensure that user inputs follow the required format before processing, reducing invalid data and improving application security.

The regular expression $\text{^aaaabb\$}$ correctly validates the required input pattern over the alphabet $\Sigma = \{a, b\}$. Regular expressions provide an efficient way to validate user inputs in web applications, improving data accuracy, security, and overall system performance.

3. In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role. Demonstrate how the regular expression a*b^* can be converted into an equivalent finite automaton (FA). Further, explain why regular expressions and finite automata are equivalent models and how they are effectively used in real-time detection systems.

ANS)

Regular Expression

a*b^*a^*b^*a*b^*

This regular expression represents zero or more a 's followed by zero or more b 's. Once a b appears, no a can appear afterward.

Equivalent Finite Automaton (FA)

States:

- q_0 – Start state (accepts zero or more a 's)
- q_1 – Accepts zero or more b 's after the a 's
- q_d – Dead (trap) state

Transition Table

Present State	Input a	Input b
---------------	---------	---------

→ *q0 q0 q1

*q1 qd q1

qd qd qd

- Start State: q0
 - Final States: q0, q1
- State Diagram**

a

+-----+

| |

-->q0* --b--> q1*

^ |

| |

+---a-- qd <--+

^ |

+---b---+

Accepted Strings: ϵ , a, aa, b, bb, ab, aab, aaabbb, etc.

Rejected Strings: ba, aba, bab, ababa, etc.

Equivalence of Regular Expressions and Finite Automata

- Every regular expression can be converted into an equivalent finite automaton (FA).
- Every finite automaton can also be converted back into a regular expression.
- Both recognize the same class of languages, called regular languages.
- Therefore, regular expressions and finite automata are considered equivalent computational models.

Applications in Real-Time Detection Systems

Regular expressions and finite automata are widely used in:

- Intrusion Detection Systems (IDS) to detect malicious network patterns.
- Compilers for lexical analysis and token recognition.
- Real-time monitoring tools to identify specific log or event patterns.
- Text search and pattern matching in editors and search engines.

Finite automata process input one symbol at a time with $O(n)$ time complexity, making them fast and suitable for real-time applications.

The regular expression a^*b^* is represented by an equivalent finite automaton with three states (including one dead state). Since regular expressions and finite automata recognize the same regular languages, they are equivalent models and are widely used for efficient pattern recognition in compilers, intrusion detection systems, and real-time monitoring tools.

4. In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language $L = \{ a^n b^n \mid n \geq 0 \}$ cannot be recognized by any finite automaton. Further, analyze the implications of this limitation in the design of modern compilers.

ANS)

Pumping Lemma for the Language $L = \{ a^n b^n \mid n \geq 0 \}$

Language

$$L = \{a^n b^n \mid n \geq 0\}$$

This language contains strings with an equal number of **a**'s followed by an equal number of **b**'s, such as:

- ϵ
- ab
- $aabb$
- $aaabbb$
- $aaaabbbb$

Proof Using the Pumping Lemma

Assume that L is regular.

According to the Pumping Lemma, there exists a pumping length p such that any string w in L with $|w| \geq p$ can be written as:

$$w = xyz$$

where:

- $|x| \leq p$ and $|y| \geq 1$
- $|y| \geq 1$
- $xy^i z \in L$ for all $i \geq 0$

Choose the string:

$$w = a^p b^p$$

Since $|x| \leq p$, the substring y contains only a 's.

Pump y with $i=2$:

$$xy^2z = a^{p+|y|} b^p$$

Now the number of a 's is greater than the number of b 's, so the string is not in L .

This contradicts the Pumping Lemma.

Therefore,

$L = \{a^n b^n \mid n \geq 0\}$ is not a regular language. $L = \{a^n b^n \mid n \geq 0\}$ is not a regular language.

Hence, no finite automaton can recognize this language.

Implications in Modern Compilers

Since finite automata cannot recognize languages like $a^n b^n$:

- Lexical analysis uses finite automata to recognize tokens such as identifiers, keywords, and numbers.
- Syntax analysis (parsing) uses context-free grammars and pushdown automata to handle nested and balanced structures.
- Compilers require a parser to check matching parentheses, nested blocks, and balanced statements, which cannot be handled by finite automata alone.

Using the Pumping Lemma, we prove that the language $L = \{a^n b^n \mid n \geq 0\}$ is not regular, so it cannot be recognized by any finite automaton. This limitation is why modern compilers use finite automata for lexical analysis and parsers (context-free grammars/pushdown automata) for syntax analysis.

5. Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules. Explain how the **closure properties of regular languages** enable the combination of multiple detection rules. Further, justify why the resulting language after applying these operations **remains regular**, ensuring efficient implementation using finite automata

ANS)

Closure Properties of Regular Languages

Closure Properties

A regular language remains regular even after applying operations such as:

- Union ($\cup$) – Combines two languages.
- Concatenation ($\cdot$) – Joins strings from two languages.
- Kleene Star ($*$) – Allows zero or more repetitions of a language.
- Intersection ($\cap$) – Finds strings common to two regular languages.
- Complement ($'$) – Accepts all strings not in the language.

These are called the closure properties of regular languages.

Combining Multiple Detection Rules

In modern systems, multiple pattern-matching rules can be combined using closure operations.

For example:

- Intrusion Detection Systems (IDS): Detect different attack signatures by combining multiple regular expressions using union.
- Spam Filtering: Match different spam keywords or patterns together.
- Input Validation: Combine rules for usernames, passwords, and email formats into a single validation process.

Thus, several detection rules can be represented as one regular language.

Why the Resulting Language Remains Regular

Regular languages are closed under union, concatenation, Kleene star, intersection, and complement. Therefore, combining two or more regular languages using these operations always produces another regular language.

As a result, the combined language can still be recognized by a Finite Automaton (FA).

Advantages in Real-World Systems

- Efficient pattern matching with linear-time ($O(n)$) processing.
- Low memory usage due to finite automata implementation.
- Easy to combine and modify multiple security rules.
- High speed and reliability for real-time applications such as intrusion detection, spam filtering, and input validation.

Closure properties allow multiple regular-language rules to be combined using operations like union, concatenation, Kleene star, intersection, and complement. Since regular languages remain regular after these operations, the resulting language can still be implemented efficiently using finite automata, making them ideal for modern security and validation systems.

SIMATS ENGINEERING ASSESSMENT TOOL

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2– 3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application

Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SIMATS ENGINEERING ASSESSMENT TOOL

2. Can enforce rules like:

- Email format
- Password rules
- Input structure